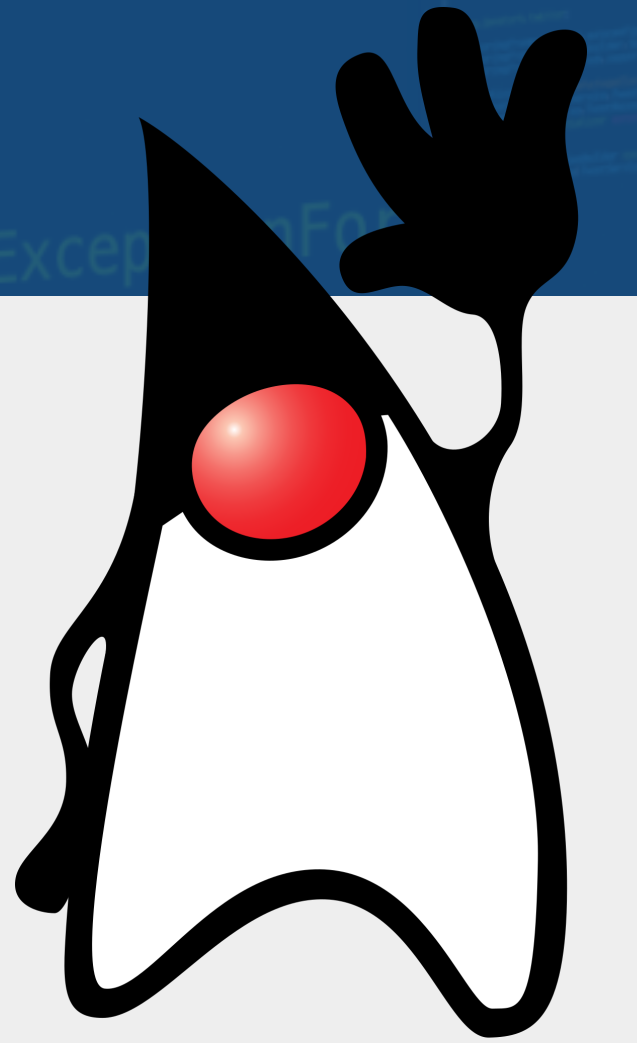


Java Programlama Dili

17. Bölüm

this

Akın Kaldıroğlu



Amaç



- Bu bölümde `this` anahtar kelimesinin kullanımları ele alınacaktır



- **this**
 - Kurucuların Birbirlerini Çağırmaları
 - this Çağrısı Yeniliği
 - this Referansı
- **Ödevler**
- **Bundan Sonra?**

this



this Anahtar Kelimesi



- Java'da **this** bir anahtar kelimedir ve iki tane kullanımı vardır:
 - Çoklu kullanılan kurucuların (overloaded constructors) birbirlerini çağırması için
 - İçinde bulunulan nesneye ulaşmak için
- Şimdi sırayla bu iki kullanımı inceleyelim



Kurucuların Birbirlerini Çağırmaları

DRY - Don't Repeat Yourself - I



- Yandaki kodun neden **DRY (Don't Repeat Yourself)** prensibine aykırı olduğunu söyleyebilir misiniz?

```
class Tree {  
    ""  
    Tree(String newType, float newHeight){  
        type = newType;  
        height = newHeight;  
    }  
  
    Tree(String newType) {  
        type = newType;  
        height = 1.0f;  
    }  
  
    Tree(float newHeight) {  
        type = "Pine";  
        height = newHeight;  
    }  
  
    Tree() {  
        type = "Pine";  
        height = 1.0f;  
    }  
}
```

DRY - Don't Repeat Yourself - II



- **DRY (Don't Repeat Yourself)** ya da **Kendini Tekrar Etme**, bir yazılım sisteminde her gerçeğin en çok bir yerde ifade edilmesi gerektiğini söyleyen bir prensiptir
- Bu prensibin bir uygulaması olarak, bir metodun hiçbir parçası başka bir metotta tekrar etmemelidir
- Dolayısıyla kuruculardaki ortak ilk değer atama ifadeleri de olabildiğince tekilleştirilmelidir

Tree



- `call.problem.Tree`

DRY - Don't Repeat Yourself - II



- Diğer üç kurucunun, en geniş ilk değer ataması yapan ilk kurucuyu çağırması, **DRY** prensibine daha uygun bir kod yapısı ortaya çıkaracaktır
- Bu sayede alanlara ilk değer atayan kodlar tekilleşecektir
- Aslında koddaki **Tree (newType, 1.0f) ;** bir derleme hatasıdır çünkü Java'da diğer kurucunun çağırılması bu şekilde olmaz, sembolik olarak böyle gösterilmiştir

```
class Tree {  
    ...  
    Tree(String newType, float newHeight){  
        type = newType;  
        height = newHeight;  
    }  
    Tree(String newType) {  
        // type = newType;  
        // height = 1.0f;  
        Tree(newType, 1.0f); //Better  
    }  
    Tree(float newHeight) {  
        // type = "Pine";  
        // height = newHeight;  
        Tree("Pine", newHeight); //Better  
    }  
    Tree() {  
        // type = "Pine";  
        // height = 1.0f;  
        Tree("Pine", 1.0f); //Better  
    }  
}
```

this ile Kurucu Çağrısı - I



- Çoklu kullanılmış kurucuların birbirlerini çağırmaları, **this** anahtar kelimesi ile yapılır
- **this**, içinde bulunulan nesneyi temsil eder ve kurucu çağrısı **this (...)** şeklinde yapılır
- **this (...)** bir kurucu metot çağrısıdır ve sadece kurucularda olabilir
- Uygun bir başka kurucu çağrısını yapacak şekilde **this (...)** 'e argüman(lar) geçilmelidir

DRY - Don't Repeat Yourself - IV



- Önce asıl çağrılan kurucu çalışacak ve akış `this (...)` ile esas atamaları yapacak kurucuya dallanacaktır
- `this (...)` çağrısı, geçilen argümanların sayısı ve tipine göre hedef kurucuyu bulur

```
void main(){  
    Tree tree = new Tree();  
}
```

```
class Tree {  
    ...  
    Tree(String newType, float newHeight){  
        type = newType;  
        height = newHeight;  
    }  
  
    Tree(String newType) {  
        // type = newType;  
        // height = 1.0f;  
        this(newType, 1.0f);  
    }  
  
    Tree(float newHeight) {  
        // type = "Pine";  
        // height = newHeight;  
        this("Pine", newHeight);  
    }  
  
    Tree() {  
        // type = "Pine";  
        // height = 1.0f;  
        this("Pine", 1.0f);  
    }  
}
```

Tree



- `call.Tree`

this ile Kurucu Çağrısı - II



- `this (...)` çağrısı ile ilgili önemli bazı noktalar vardır:
- `this (...)` çağrısı sadece kurucularda yapılabilir, başka bir yerde mesela bir metotta `this (...)` çağrısı yapılamaz
- `this (...)` çağrısı ile sadece kurucular çağrılabilir çünkü `this` isminde bir metot yazılamaz

this ile Kurucu Çağrısı - II



- `this (...)` kendisini çağıramaz, yani özyinelemeli (recursive) olamaz, derleyici hata verir
- Aksi takdirde JVM, **StackOverflowError** hatası ile çalışmasına son verirdi
- İki kurucu `this (...)` ile karşılıklı olarak birbirlerini çağıramazlar
- Bu da özyinelemeli çağrı olur

this ile Kurucu Çağrısı - IV



- **this (...)** çağrısı bir kurucudaki ilk cümle olmalıdır, aksi takdirde derleyici hata verir
- Çünkü önce nesnenin oluşturulması tamamlanmalıdır; bundan önce nesneden bilgi ya da hizmet almak türünden hiçbir iş yapılmamalıdır
- Dolayısıyla Java derleyicisi **this (...)** çağrısından önce çalışacak bir satır olup olmadığını kontrol eder
- Bunun bir sonucu olarak da bir kurucuda sadece bir tane **this (...)** çağrısı yapılabilir

this ile Kurucu Çağrısı - V



- Geliştirici, `this (...)` çağrısının olmadığı kurucularda da nesnenin oluşturulması tamamlanmadan başka bir iş yapmamalıdır
- Ama derleyici bu durumu anlayamaz ve hata vermez!
- Bunu daha önce de konuşmuştuk:
 - Önce nesnenin oluşturulması tamamlanmalı ve nesne istenilen duruma gelmelidir; bundan önce nesneden bilgi ya da hizmet almak türünden hiçbir iş yapılmamalıdır, aksi takdirde beklenmedik sonuçlar olabilir

ThisProblems

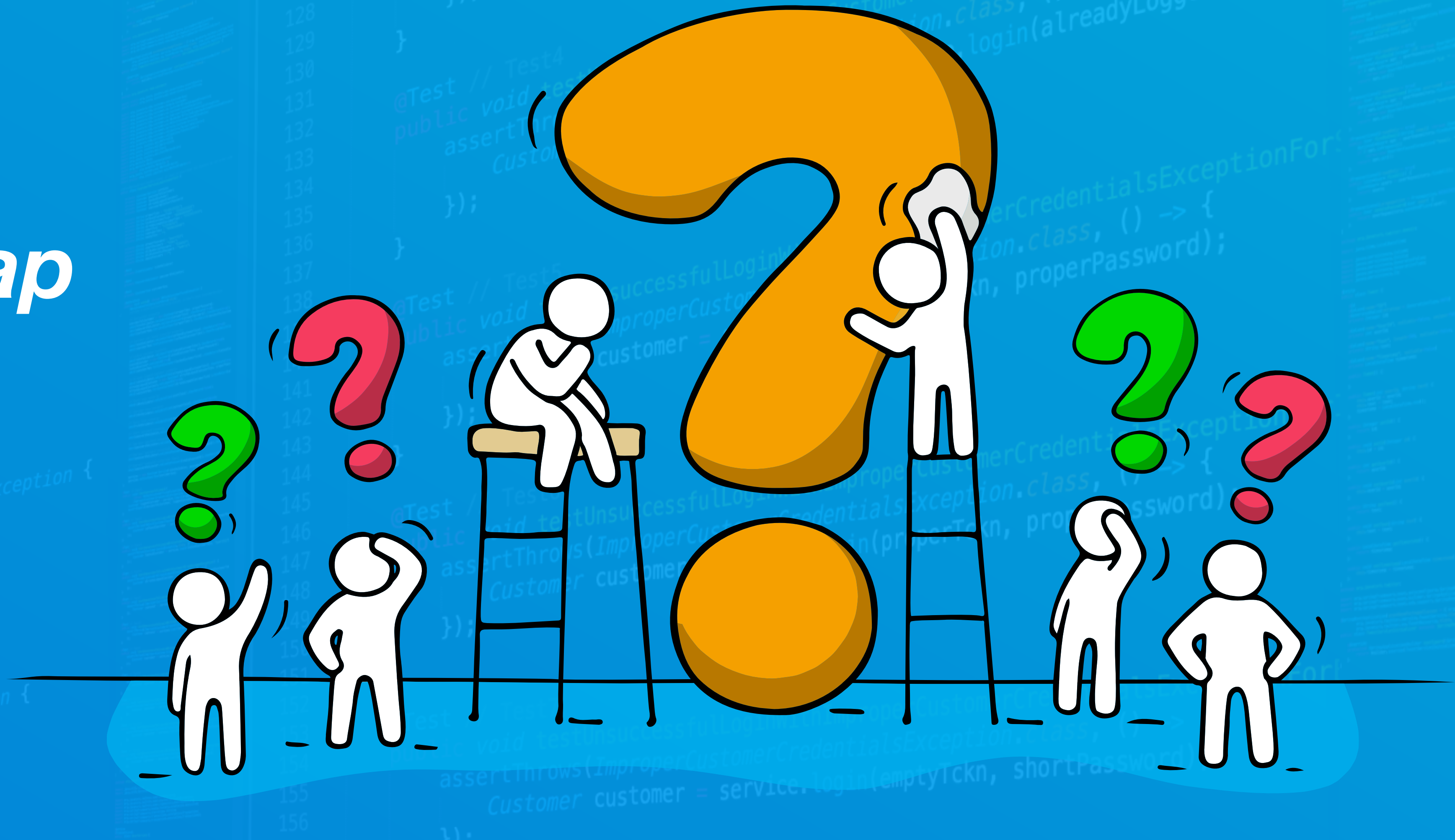


- `call.problem.ThisProblems`
- Bu örnekte `this()` çağrısı ile ilgili şu durumları gözlemleyin:
 - İlk satırda olması gerektiğini
 - Özyinelemeli çağrılar
 - Bir kurucuda birden fazla çağrıyı
 - Karşılıklı çağrılar



- `call.problem.Tree`
- Bir kurucuda `this()` kullanılmadığında bile nesnenin oluşturulmasına öncelik verip, yapılacak işlerin bunu takip etmesi gerektiğini fark edin

Soru ve Cevap Zamanı!





this Çağrısı Yeniliği

this Çağrısı Yeniliği - I



- “*this (...) çağrısı bir kurucudaki ilk cümle olmalıdır, aksi takdirde derleyici hata verir*” dendi
- Bu kural Java 25 ile değiştirildi ya da daha doğru ifade ile yumuşatıldı
- JDK 22’de (JEP 447) teklif edilip, 23'te (JEP 482) ve 24'te (JEP 492) ön izlemesi yapılan ve 25'te kalıcı hale gelen **JEP 513: Flexible Constructor Bodies** (<https://openjdk.org/jeps/513>) ile artık `this` (ve sonraki eğitimde ele alınacak `super`) çağrılarının bir kurucuda ilk yapılan iş olması gerektiği ile ilgili kural gevşetildi

this Çağrısı Yeniliği - II



- Bu kuralla `Tree` sınıfının kurucularında, geçilen argümanların kontrol edilmesi gerektiğinde, bu ancak içinde `this` çağrısı olmayan kurucuda yapılabilir

```
// Java 21 code
class Tree {
    ...
    Tree(String newType, float newHeight) {
        if (newType != null && newHeight ≥ 0) {
            type = newType;
            height = newHeight;
        }
    }

    Tree(String newType) {
        // Can't check if newType is null
        this(newType, 1.0f);
    }

    Tree(float newHeight) {
        // Can't check if newHeight is not negative
        this("Pine", newHeight);
    }
    ...
}
```

this Çağrısı Yeniliği - III



- Bu yeni kuralla kurucuda yapılacak **this** çağrısından önce geçilen argümanlarla ilgili kontroller yapılabilir
- Yandaki kod JDK 21 ile derlendiğinde hata verirken JDK 25 ile derlendiğinde hata vermez

```
// Java 25 code
class Tree {
    ...
    Tree(String newType, float newHeight) {
        if (newType != null && newHeight ≥ 0) {
            type = newType;
            height = newHeight;
        }
    }

    Tree(String newType) {
        if (newType == null)
            throw new IllegalArgumentException("...");
        this(newType, 1.0f);
    }

    Tree(float newHeight) {
        if (newHeight ≤ 0)
            throw new IllegalArgumentException("...");
        this("Pine", newHeight);
    }

    Tree() {
        String newType = "Pine";
        float newHeight = 1.0f;
        this(newType, newHeight);
    }
}
```

this Çağrısı Yeniliği - IV



- **this** çağrısından önce nesne değişkenlerinin değerini okumak hâlâ mümkün değildir çünkü nesnenin oluşturulması henüz tamamlanmış değildir ama değer atanabilir
- Yandaki kod JDK 21 ile derlendiğinde hata verirken JDK 25 ile derlendiğinde hata vermez

```
// Java 25 code
class Tree {
    ...
    Tree(String newType, float newHeight) {
        if (newType != null && newHeight >= 0) {
            type = newType;
            height = newHeight;
        }
    }

    Tree(String newType) {
        if (newType == null)
            throw new IllegalArgumentException("...");
        this(newType, 1.0f);
    }

    Tree(float newHeight) {
        if (newHeight <= 0)
            throw new IllegalArgumentException("...");
        this("Pine", newHeight);
    }

    Tree() {
        String newType = "Pine";
        float newHeight = 1.0f;
        //System.out.println(height); // Not OK
        height = 5; // OK
        this(newType, newHeight);
    }
}
```

Tree



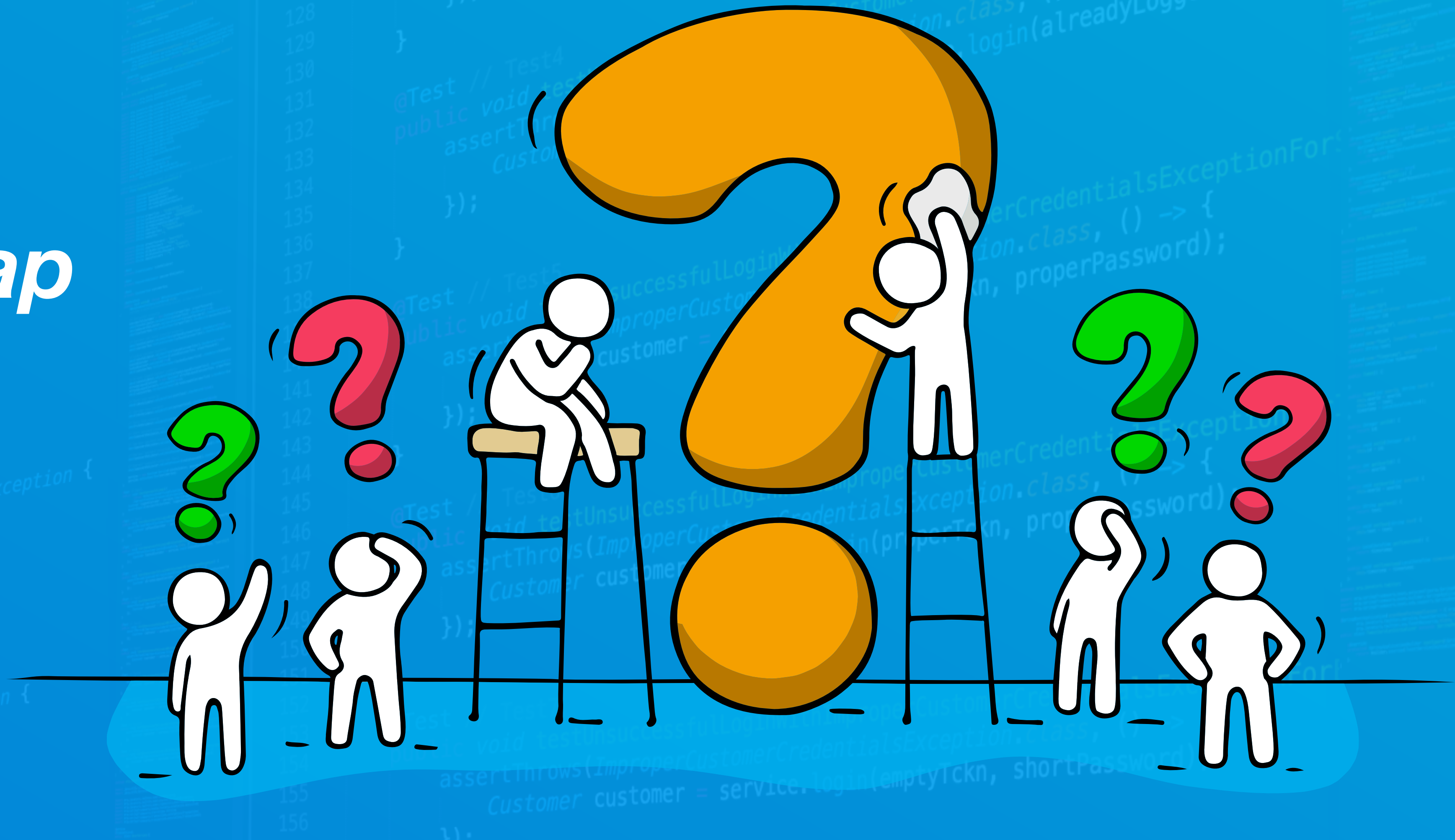
- `call.newThis.tree.Tree`
- **Tree** sınıfını önce Java 25 ile sonra da komut satırında Java 21 ile derleyin

UserProfile



- `call.newThis.user.UserProfile`

Soru ve Cevap Zamanı!





this
Referansi

this - I



- **this** anahtar kelimesinin, kurucuların birbirlerini çağırmak için nasıl kullanıldığını gördük
- Bu kullanım, **this (...)** şeklindeki metot çağrısıydı
- Aslen **this**, içinde bulunan nesneyi gösteren bir referanstır
- Tipik olarak kurucular ve metotlarda kullanılır

this - II



- **this** herhangi bir sebeple, içinde bulunulan nesneye erişmek için kullanılır
- **this** ile nesnenin herhangi bir alanına ve metoduna ulaşılır
- **this**'in zorunlu olan ve olmayan kullanımları yanında gereksiz ya da hatalı kullanımları da vardır

this - III



- Yandaki kullanımların hiç birisinde **this** zorunlu değildir
- Çünkü ulaşılan değişkenler ve metotlar zaten içinde bulunan nesneye aittir
- Yandaki iki kod birbirinin aynısıdır, dolayısıyla üstteki durumda **this** *gereksiz (redundant)* bir kullanımdır

```
class ThisDemo {  
    int i = 0;  
    int k = 5;  
    int j = 2 * this.k;  
  
    void aMethod() { this.printI();}  
  
    void printI() {  
        System.out.println(this.i);  
    }  
}
```

```
class ThisDemo {  
    int i = 0;  
    int k = 5;  
    int j = 2 * k;  
  
    void aMethod() {printI();}  
  
    void printI() {  
        System.out.println(i);  
    }  
}
```



- **this** kullanımının zorunlu olduğu tipik durum şudur:
 - Bir kapsamda (scope) aynı isimde bir yerel bir de nesne değişkeni varsa, doğrudan erişimde yerel değişkene ulaşılır
 - Bu durumda nesne değişkenine erişmek için, *nesnenin* anlamında **this** referansı kullanılır
 - **this** kullanmadan nesne değişkenine ulaşamaz, yerel değişkene erişilir

this - V



- Kurucu ve `set` metotlarında anlaşılmayı kolaylaştırmak için nesne değişkeniyle, ona ilk değer vermek için geçilen parametreye aynı isim verilir
- Burada nesnenin değişkenine ulaşmak için **this** zorunludur aksi halde yerel (local) değişkene ulaşılır ve totoloji oluşur

```
class ThisDemo {  
    int i = 0;  
    int k = 5;  
    int j = 2 * k;  
  
    ThisDemo(int i, int k, int j){  
        // i = i; // Tautology, always true!  
        this.i = i;  
        this.k = k;  
        this.j = j;  
    }  
  
    void setI(int i) {  
        this.i = i;  
    }  
}
```

this - VI



- Nesne alanları tanıtılırken `this` kullanımı bir derleme hatasıdır
- Ama ilk değer ifadesinde `this` gereksiz de olsa kullanılabilir

```
class ThisDemo {  
    int this.i = 0; // Error!  
  
    int k = 5;  
    int j = 2 * this.k; // Ok!  
    ...  
}
```

this - VII



- Ayrıca **this** referansına atama yapılamaz
- Aşağıdaki gibi bir kod **this**'in içinde bulunan nesne dışında başka bir nesneyi göstermesi anlamına gelir ki derleme hatasıdır

```
class ClassA{  
    void f(){  
        this = new ClassA(); // Error!  
    }  
    ...  
}
```

ThisDemo ve ThisTree



- `ref.ThisDemo`
- `ref.ThisTree`
- Bu iki örnekte gerekli, gereksiz ve hatalı `this` kullanımları vardır

this - VIII



- Örnekteki `grow()` metodunda `this` metottan geri döndürülmektedir
- `this`'in tipi, içinde bulunduğu nesnenin tipidir

```
class Tree {  
    ...  
    Tree copy() {  
        Tree copy = new Tree(type, height);  
        return copy;  
    }  
  
    Tree grow() {  
        height++;  
        return this;  
    }  
}
```



- `ref.Tree`
- Bu örnekte `Tree` sınıfındaki `copy` ve `grow` metotlarını inceleyin
- Bu metotların dönüş değerleri ile tipleri arasındaki uyuma dikkat edin
- Bu metotların üzerinde çağrıldıkları nesneler ile döndürdükleri referansların gösterdiği nesnelerin aynı olup olmadıklarını inceleyin

this - IX



- **this** metoda geçilebilir

```
class Customer {  
    int id;  
    String name;  
    CreditCard cc;  
  
    Customer(int id, String name){  
        this.id = id;  
        this.name = name;  
    }  
  
    void setCc(CreditCard cc) {  
        this.cc = cc;  
    }  
}
```

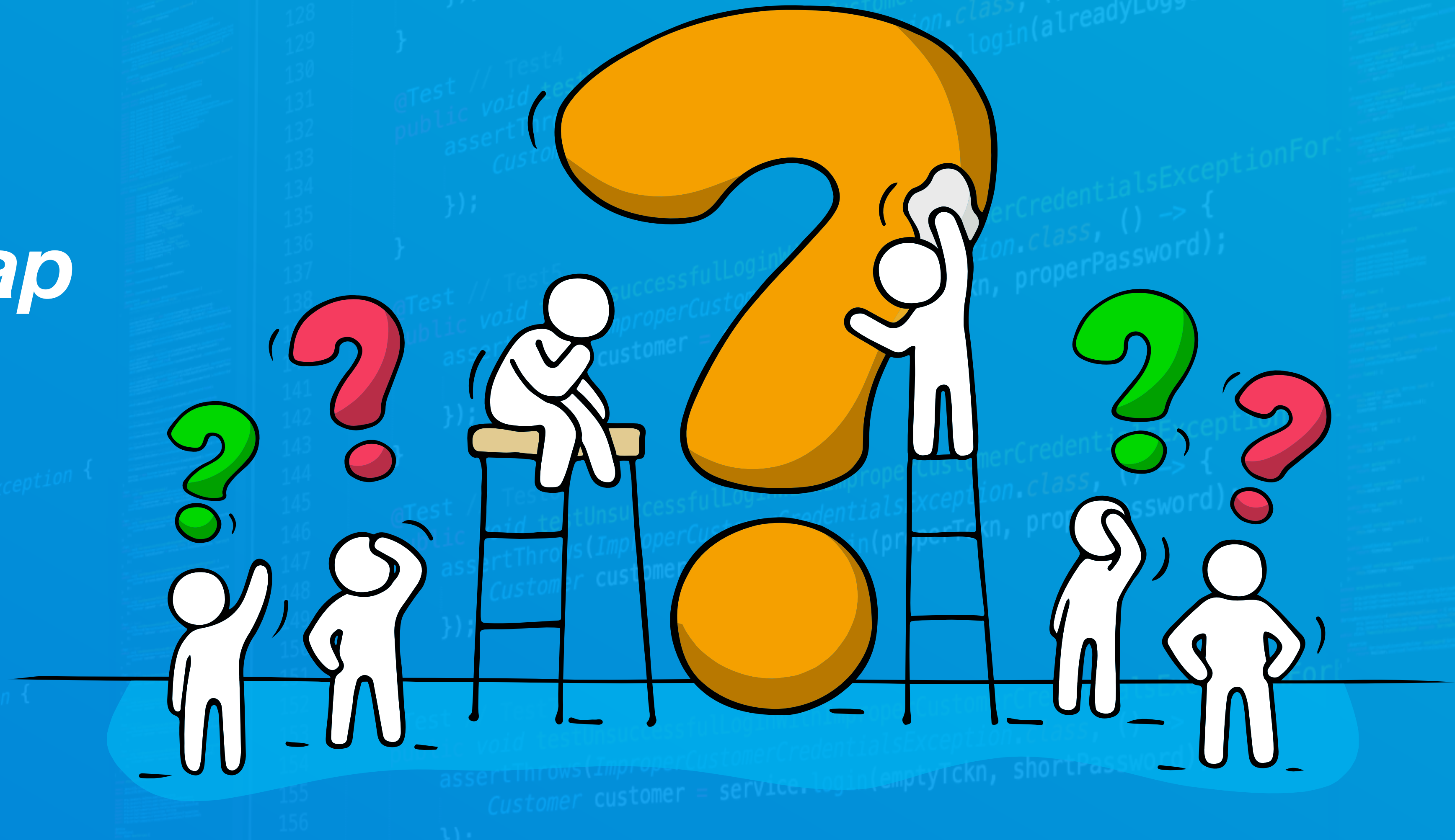
```
class CreditCard {  
    String no;  
    double balance;  
    Customer owner;  
  
    CreditCard(String no, double balance){  
        this.no = no;  
        this.balance = balance;  
    }  
  
    CreditCard(String no,  
                double balance, Customer owner){  
        this.no = no;  
        this.balance = balance;  
        this.owner = owner;  
        owner.setCc(this);  
    }  
  
    void setOwner(Customer owner){  
        this.owner = owner;  
    }  
}
```

```
class BankTest {  
    public static void main(String[] args) {  
  
        Customer c1 = new Customer(1, "Ahmet");  
        CreditCard cc1 = new CreditCard("1", 1000);  
        cc1.setOwner(c1);  
        c1.setCc(cc1);  
  
        // Alternatively  
        Customer c2 = new Customer(2, "Kemal");  
        CreditCard cc2 = new CreditCard("2", 2500, c2);  
    }  
}
```



- `ref.bank`
- `Customer`
- `CreditCard`
- Bu örnekte `CreditCard` sınıfının kurucusunda, içinde bulunulan `CreditCard` nesnesinin, `Customer`'in `setCC()` metoduna nasıl geçildiğini inceleyin

Soru ve Cevap Zamanı!



Ödevler





1. Bir önceki bölümde modellediğiniz köpek ve sahibiyle ilgili oluşturduğunuz sınıflara kurucularına **this ()** çağrıları ekleyin
 - a. Sonra test sınıfını yazıp **main** metodunda nesnelerini oluşturup değerler atayın



2. Bir önceki bölümdeki iki boyutlu bir ortamda bir noktayı temsil eden sınıfın kurucularına `this ()` çağrıları ekleyin
 - a. Test sınıfının `main` metodunda, birisi merkezi (origin) temsil eden 3 noktayı kurucu çağrılarıyla oluşturup üzerlerinde metot çağrıları yapın



3. Bir önceki bölümde ele aldığınız **Circle/Daire** sınıfındaki argümansız kurucuya **this ()** çağrısı ekleyerek yarıçapı 10 olan daireyi oluşturmasını sağlayın
 - a. Sınıftaki argüman alan kurucuda **this** referansını kullanın
 - b. **CircleTest/DaireTest** sınıfında her iki kurucuyu da çağırarak nesneler oluşturup çevre ve alanlarını hesaplatın



4. Bir önceki bölümde modellediğiniz iki boyutlu bir ortamda bir nokta soyutlamasını ifade eden sınıfınıza kurucularına **this ()** çağrısı ekleyin
 - a. Sınıftaki argüman alan kurucuda **this** referansını kullanın
 - b. Test sınıfının **main** metodunda, birisi merkezi (origin) temsil eden 3 nokta oluşturup üzerlerinde metot çağrıları yapın



5. Bir önceki bölümde modellediğiniz elektrikle çalışan bir lambayı ya da ampülü (bulb) temsil eden sınıfa kurucularına **this ()** çağrısı ekleyin
 - a. Sonra test sınıfını yazıp **main** metotta nesnelerini oluşturup üzerinde metot çağrıları yapın



6. **SmartDevice** isimli bir sınıf oluşturun ve üzerine **brand**, **model** ve **price** alanlarını koyun ve aşağıdaki kurucuları ekleyin:
- Üç parametrenin tümünü alan bir kurucu
 - Yalnızca **brand** ve **model** alan bir yapıcı, bu durumda **price**'in değeri \$100,00 olarak belirlensin
 - **brand** değerini “Unknown” ve **model** değerini “Generic” olarak atayan parametresiz bir kurucu
 - İkinci ve üçüncü kurucularda **this ()** çağrısını kullanın



7. Bir önceki sorudaki `SmartDevice` isimli bir sınıf kurucularındaki `this ()` çağrılarından önce nesnenin alanlarına ulaşmaya çalışın, hem okuyun hem atama yapın
- Mümkünse bu kodu hem Java 25 hem de öncesindeki sürümlerle örneğin Java 21 ile derleyin



8. Bir oyunda kullanılacak ve `health`, `mana`, and `username` alanlarına sahip bir `Player` sınıfı oluşturun ve uygun kurucu ve setter metotlarını koyun, mümkün olan yerlerde `this` referansını kullanın
 - a. Kurucu ve setter metotlarında `this` referansını kullanın



9. **Hero** and **Potion** isimli, sağlıkla ilgili bir oyunda kullanılacak iki sınıf oluşturun

1. **Hero** sınıfına **usePotion(Potion p)** metodunu ekleyin
2. **Potion** sınıfına, nesnenin sağlığını iyileştiren **applyEffect(Hero h)** metodu ekleyin
3. **Hero** sınıfının **usePotion()** metodunda, geçilen **Potion** nesnesi üzerinde **applyEffect()** metodunu çağırın



10. Bir önceki bölümde örnek olarak verilen `overloaded.Employee` sınıfının kurucularına da `this ()` çağrılarını koyun
- a. `Test` sınıfında her kurucuyu çağırarak `Employee` nesneleri oluşturun



11. Şifreyi temsil eden bir `Password` sınıfı modelleyin. Şifrenin uzunluğunun en başta belirlendiğini ve sonrasında karakter ekleyerek oluşturulduğunu düşünün. Bu amaçla şifreye, uzunluğu sayısınca geçilen karakterleri ekleyerek oluşturan `PasswordBuilder` gibi bir sınıfa ihtiyaç olacaktır. `PasswordBuilder`'i kullanarak aşağıdaki gibi `Password` nesnesini oluşturacak şekilde bu iki sınıfı geliştirin:

```
Password password = new PasswordBuilder().setLength(length) .  
    add(ch1) .add(ch2) .add(ch3)...build();
```

a. Test sınıfının `main` metodunda gerekli testleri yazın

Bundan Sonra?



Bundan Sonra?



- Bundan sonra `static` anahtar kelimesini göreceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

